

Drake AI Enterprise Platform

Simple but exact technical stack and working explanation for every module and section.

1. Overall Stack

Area	Stack Used	Simple Explanation
Frontend	React 18, TypeScript, Vite, Tailwind CSS, React Router, Zustand, TanStack Query, Axios, Plotly, Recharts	This is the user interface. It shows pages, buttons, upload screens, charts, tables, and module results.
Backend	FastAPI, Python, Pydantic, SQLAlchemy, Alembic, Uvicorn	This is the API server. It receives files and requests from the UI, runs calculations/models, and sends results back.
Database	SQLite local default, PostgreSQL 15 in Docker	Stores users, projects, wells, curves, jobs, and related records.
Cache/session	Redis with local fallback	Keeps temporary LAS/session data so modules can reuse uploaded files.
Storage	Local uploads folder, optional MinIO/S3-compatible storage	Stores uploaded LAS, seismic, result, and export files.
AI/ML libraries	scikit-learn, optional XGBoost, TensorFlow/Keras, PyWavelets, NumPy, pandas, SciPy, lasio, segyio	These libraries parse data, calculate statistics, train models, run predictions, and enhance seismic data.

Quick Note

Some sections use machine learning models. Some sections do not. Crossplot and Histogram are statistical/visual tools, so the PDF clearly says no ML model for them.

2. UI Sections

UI Section	Stack Used	How It Works
Login/Auth UI	React, Zustand token state, Axios	User logs in. The token is saved in frontend state and used for protected API calls.
Main layout	React Router, MainLayout, Sidebar, TopBar, Ribbon, StatusBar	Creates the workspace shell and lets users move between modules.
Project pages	React pages, local project storage, Axios APIs	Shows projects and keeps selected project context for modules.
Data management	React upload UI, Axios, local storage helpers	Lets users manage uploaded files and saved module outputs.
Charts and plots	Plotly.js, Recharts	Renders log tracks, crossplots, histograms, seismic heatmaps, prediction curves, and trend charts.
Uploads	React Dropzone, Axios multipart upload	Uploads LAS, SEG-Y, NPY, CSV, TXT, XLSX files to backend endpoints.
Notifications	React Hot Toast, lucide-react icons	Shows success/error messages and icon controls.

3. Core Backend Modules

Module / Section	Stack Used	Model / Method Used	How It Works in Simple Words
Auth	FastAPI, SQLAlchemy, JWT, bcrypt	No ML model	Checks email/password, creates a secure token, and protects private API routes.
Projects	FastAPI, SQLAlchemy, Pydantic	No ML model	Creates, lists, opens, and manages project records.
Wells	FastAPI, SQLAlchemy	No ML model	Stores and returns well metadata like name, field, depth, status, and related curves.
Curves	FastAPI, SQLAlchemy, JSON curve data	No ML model	Returns curve names and curve depth/value arrays for plotting and analysis.
Files	FastAPI UploadFile, lasio, pandas, local uploads	No ML model	Reads uploaded files, extracts curves, stores file data, and prepares it for modules.
Reports	FastAPI, fpdf2, openpyxl, pandas, FileResponse	No ML model	Exports results as report files, spreadsheets, CSV/JSON, or downloadable outputs.

4. Petrophysics Modules and Sections

Petrophysics Section	Stack Used	Model / Method Used	How It Works in Simple Words
LAS upload/session	FastAPI, lasio, pandas, NumPy, Redis/local cache	No ML model	Reads the LAS file, cleans depth/curve values, saves it as a session, and returns available curves.
Log visualization	React, Plotly, backend LAS session data	No ML model	Plots selected well logs against depth so users can view curve behavior.
Crossplot	pandas, NumPy, Plotly scattergl	No ML model. Uses filtering and correlation.	Takes two selected curves, removes invalid rows, applies depth/log-scale filters, calculates correlation, and draws an X-Y scatterplot.
Histogram	NumPy histogram, SciPy gaussian_kde, SciPy stats	No ML model. Statistical analysis only.	Takes one curve, builds histogram bins, KDE curve, mean, median, percentiles, skewness, kurtosis, outliers, and quality score.
Missing log prediction	scikit-learn ExtraTreesRegressor, RandomForestRegressor, GradientBoostingRegressor	Extra Trees, Random Forest, or Gradient Boosting	Learns from available valid log samples and predicts missing values for the selected target curve.
AI parameter prediction	RandomForestRegressor, optional XGBRegressor, GradientBoostingRegressor, DecisionTreeRegressor, formulas	Empirical, Random Forest AI, XGBoost AI, Gradient Boosting AI, Decision Tree AI	Calculates VSH, porosity, water saturation, permeability, lithology, confidence, and P10/P50/P90 results.
AI uncertainty	Prediction engine, NumPy, pandas, ensemble spread	Random Forest spread, XGBoost/boosting bootstrap spread, percent fallback	Builds uncertainty bands around prediction results so users see low, middle, and high cases.
Facies classification	StandardScaler, KMeans, MiniBatchKMeans, RandomForestClassifier	KMeans/MiniBatchKMeans or RandomForestClassifier	Groups log patterns into facies. If labels exist, it trains Random Forest; otherwise it clusters curves automatically.
Formation tops	SciPy savgol_filter, find_peaks, gradients, RandomForestClassifier optional	Gradient peak detection or supervised RandomForestClassifier	Finds strong curve changes with depth and marks them as possible formation boundaries.
Auto splicer	Depth matching, overlap merge job logic	No external trained ML model	Aligns overlapping log intervals by depth and merges them into a cleaner continuous result.

5. Domain Modules

Module	Stack Used	Model / Method Used	How It Works in Simple Words
Drake GPT	FastAPI, Anthropic SDK, SQLAlchemy well context	claude-sonnet-4-20250514; fallback Drake GPT Local	Builds a well-specific prompt from curves, tops, and jobs. Claude answers if API key exists; otherwise local rules answer common petrophysics questions.
AI jobs	FastAPI, SQLAlchemy job records, backend/ml/petrophysics.py	Returned labels include LSTM + Random Forest Ensemble, Random Forest Classifier, CNN boundary detector, Gradient Boosting, Random Forest + FZI, Archie + Neural Network Sw	Creates background-style analysis jobs, tracks status/progress, and stores predicted curve/result metadata.
Seismic upload/inspect	FastAPI, segyio, NumPy, pathlib	No ML model	Uploads seismic files and reads trace count, sample count, sample interval, and data shape.
Seismic enhancement	TensorFlow/Keras Conv1D, PyWavelets, RandomForestRegressor, FFT, NumPy	2D Conv1D U-Net, 3D PyWavelets + RandomForest, Conv1D autoencoder, FFT fallback	Enhances low/high frequency seismic data and returns original, enhanced, difference, metrics, spectrum, and optional SEG-Y output.
Production intelligence	pandas, NumPy, RandomForestClassifier	RandomForestClassifier for artificial lift failure; rule fallback	Reads production data, calculates water cut/GOR/downtime, predicts failure risk when possible, ranks optimizer actions, decline, health, loss, and workover candidates.
Geothermal screening	Custom LAS parser, pandas, NumPy, matplotlib	No trained ML model. Rule/calculation based.	Calculates geothermal gradient, temperature, VSH, porosity, permeability, reservoir quality, hot zones, heat flow, and map/export data.
CCUS screening	Custom LAS parser, rule engine, Plotly data, XLSX writer	No ML model. Cutoff/rule-based scoring.	Uses log-derived PHIE, VSH, permeability, GR, RHOB, NPHI, RT, and user cutoffs to rank CO2 storage and seal zones.
Standalone Drake_Uncertainty	Flask, Jinja, Plotly, lasio, pandas, NumPy, SciPy, scikit-learn, optional XGBoost	RandomForestRegressor, XGBRegressor, GradientBoostingRegressor, DecisionTreeRegressor, RandomForestClassifier, empirical formulas	Separate dashboard app for LAS upload, prediction, uncertainty, visualization, and exports.

6. Exact Model and Algorithm List

Model / Algorithm	Used In	Simple Purpose
claude-sonnet-4-20250514	Drake GPT	AI chat/copilot answers using well context.
Drake GPT Local	Drake GPT fallback	Rule-based answers when Claude is not available.
ExtraTreesRegressor	Missing log prediction	Predicts missing curve samples.
RandomForestRegressor	Missing logs, AI prediction, seismic low-frequency	Regression and ensemble spread for predictions/enhancement.
GradientBoostingRegressor	Missing logs, AI prediction	Boosted regression for curve/parameter prediction.
XGBRegressor	AI prediction and standalone uncertainty app	Optional boosted model for predictions and uncertainty.
DecisionTreeRegressor	AI prediction and standalone app	Tree-based prediction option.
KMeans / MiniBatchKMeans	Facies classification	Groups similar log patterns without labels.
RandomForestClassifier	Facies, formation tops supervised mode, production failure	Classification for labels or failure risk.
SciPy find_peaks + savgol_filter	Formation tops	Finds strong curve breaks.
SciPy gaussian_kde	Histogram	Draws smooth distribution curve; not ML.
TensorFlow Conv1D U-Net	Seismic 2D	Deep learning trace enhancement.
TensorFlow Conv1D autoencoder	Seismic high frequency	Enhances selected frequency bands.
PyWavelets + RandomForest	Seismic low frequency	Enhances wavelet low-frequency coefficients.
FFT fallback	Seismic fallback	Simple frequency-band boost when advanced models fail.
Rule/formula scoring	CCUS, geothermal, empirical prediction, production fallback	Deterministic calculations from logs and thresholds.

7. Code References

Area	Main Code Files
Frontend	frontend/src/App.tsx; frontend/src/pages/UIOnlyModulePage.tsx; frontend/src/components/*; frontend/src/services/api.ts
Backend entry	backend/app/main.py; backend/app/core/config.py; backend/app/core/security.py; backend/app/core/database.py
Petrophysics	backend/app/api/petrophysics.py; backend/app/ml/drake_prediction_standalone.py; backend/app/ml/drake_uncertainty*.py
Facies/tops	backend/app/ml/mini_petrophysics_toolbox.py
Seismic	backend/app/api/seismic.py; backend/app/ml/seismic_enhancer.py
Production	backend/app/api/production.py; backend/app/ml/production_intelligence.py
Geothermal	backend/app/api/geothermal.py; backend/app/ml/geothermal.py
CCUS	backend/ccus/router.py; backend/ccus/core.py
GPT	backend/app/api/gpt.py
Standalone app	Drake_Uncertainty/app.py; Drake_Uncertainty/utlis/ml_utils.py; templates; static

Final Note

This report is based on the code present in this repository. When a module does not use ML, it is marked as no ML model. When the code returns demo/result labels, the report lists those labels exactly as code behavior.